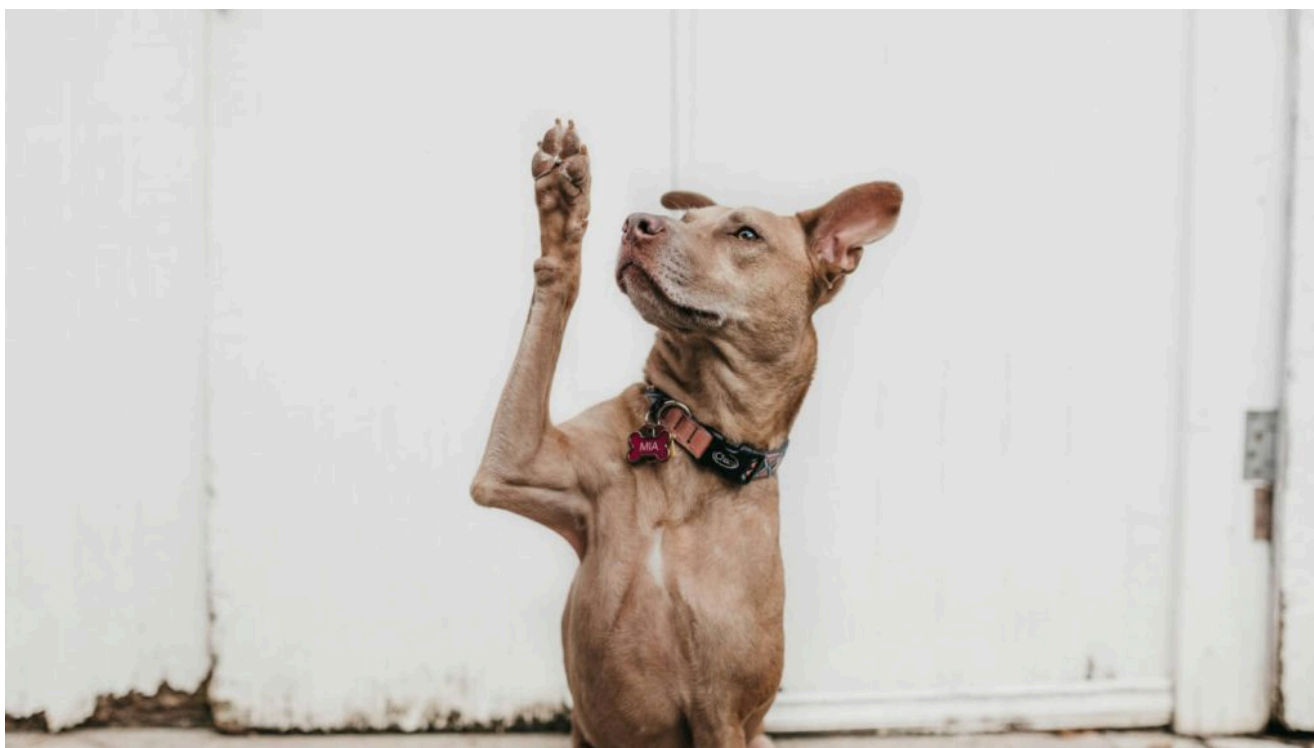


ДАННЫЕ / БАЗЫ ДАННЫХ

Как писать SQL-запросы

Узнайте, как использовать SELECT, FROM, JOIN, WHERE, GROUP BY, HAVING, ORDER BY, OFFSET и FETCH для извлечения данных с помощью SQL.

27 марта 2024 года, 8:28 [Джеральд Венцль](#)



Изображение предоставлено Камиллой Баттони с Unsplash.

[Oracle](#) спонсировала этот пост.

SQL — это декларативный, похожий на английский язык предметно-ориентированный язык для запросов, анализа и обработки данных. SQL появился в **реляционных базах данных**, но с тех пор получил широкое распространение в других областях. SQL считается декларативным языком, то есть пользователи объявляют, какие результаты им нужны, а не как их получить (последний подход характерен для императивных языков программирования, таких как C, Java и **Python**). Благодаря этому, а также возможности читать SQL-запросы почти как предложения на английском языке, SQL считается одним из лучших высокоуровневых декларативных языков программирования для анализа данных благодаря своему **простому в освоении синтаксису**. В SQL есть различные языковые элементы, которые на высоком уровне можно разделить на **запросы** и **манипуляции с данными**. В SQL-запросах используется оператор `SELECT`, а в SQL для манипуляций с данными — операторы `INSERT`, `UPDATE`,

DML.

Определение SQL-запросов

SQL-запросы — это, пожалуй, самые распространенные операции в SQL, поскольку они позволяют пользователям извлекать и анализировать данные из одной или нескольких таблиц. SQL-запросы состоят из следующих элементов:

1. `SELECT` и `FROM`
2. `SELECT` без `FROM`
3. `JOIN`
4. `WHERE`
5. `GROUP BY`
6. `HAVING`
7. `ORDER BY`
8. `OFFSET`
9. `FETCH`
10. `OFFSET` and `FETCH`

Оператор `SELECT` включает в себя несколько элементов, но обязательными являются только первые два: `SELECT` и `FROM`. Однако в некоторых базах данных, включая **Oracle Database** и MySQL, предложение `FROM` является необязательным, если `SELECT` относится только к автономным выражениям, таким как `SELECT 1; SELECT sysdate; и SELECT my_function();`. В этих случаях данные не извлекаются из таблицы, поэтому `FROM` не требуется. Необязательные компоненты обозначаются [] вокруг них.

```
1 SELECT <expressions>
2     FROM <table or sub-query>
3     [ JOIN <to other table> ON <join condition> ]
4     [ WHERE <predicates> ]
5     [ GROUP BY <expressions>
6       [ HAVING <predicates> ] ]
7     [ ORDER BY <expressions> ]
8     [ OFFSET ]
9     [ FETCH ]
```

Распространенное заблуждение заключается в том, что эти компоненты выполняются в том же порядке, в котором они указаны в запросе. Это не так, поскольку компонент `SELECT` обрабатывается после предложения `HAVING`.
«е приведен порядок обработки предложений и их назначение:

обычно это относится к таблице, но может включать и подзапрос (другой `SELECT` запрос, который выступает в качестве источника входных данных для текущего запроса).

2. `JOIN` : Определяет правила объединения нескольких таблиц. Предложение `JOIN` является частью предложения `FROM` и объединяет данные из нескольких таблиц в один набор данных. Это один из фундаментальных операторов реляционной модели, позволяющий объединять различные отношения в один набор. Предложение `JOIN` допускает условия соединения, которые гарантируют, что соединяются только строки, которые логически принадлежат друг другу (строки с соответствующими связями первичный ключ → внешний ключ). Для объединения нескольких таблиц в набор данных можно указать несколько предложений `JOIN` . Поскольку `JOIN` предложение является частью `FROM` предложения, оно не может быть указано без предшествующего `FROM` оператора в запросе.
3. `WHERE` : Фильтрует строки, возвращаемые запросом. Предложение `WHERE` фильтрует набор данных на основе предоставленных *предикатов* или условий фильтрации и отбрасывает все строки, которые им не соответствуют. Это позволяет сузить область поиска, например, чтобы получить все `countries` с континента `Europe` вместо всех стран мира.
4. `GROUP BY` : объединяет (или группирует) строки с общими значениями в указанном столбце (столбцах) в одну строку. Полученную строку иногда называют *группировочным набором*. Поскольку строки с общими значениями объединяются в одну строку, количество строк будет равно количеству уникальных значений. Для значений столбцов, не указанных в `GROUP BY` , требуются агрегатные функции в предложении `SELECT` , чтобы объединить эти значения в группу.
5. `HAVING` : фильтрует строки, полученные в результате выполнения предложения `GROUP BY` . Следовательно, он является частью `GROUP BY` и не может быть использован без предшествующего `GROUP BY` оператора в запросе.
6. `SELECT` : Определяет список столбцов и выражений, которые должны отображаться в выходных данных результата запроса. Предложение `SELECT` вычисляет любые выражения и определяет список столбцов, которые должны быть возвращены или *спроецированы* в качестве результата запроса.
7. `ORDER BY` : Определяет, какой столбец (столбцы) использовать для сортировки результирующих данных и в каком направлении их сортировать (по возрастанию или по убыванию). Если `ORDER BY` не указан, порядок строк, возвращаемых SQL-запросом, не определен.

9. `FETCH` : Указывает количество строк, которые нужно вернуть из результата.

Использование SQL-запросов

Теперь, когда вы знаете, что означают различные части SQL-запросов, вы можете начать их использовать. Вы можете выполнять эти упражнения, используя модель данных из моего [репозитория на GitHub](#).

SELECT и FROM

В простейшей форме SQL-запрос состоит из `SELECT` и `FROM` частей:

```
1 SQL> SELECT *
2     2*   FROM regions;
3
4 REGION_ID  NAME
5 -----
6 AF         Africa
7 AN         Antarctica
8 AS         Asia
9 EU         Europe
10 NA        North America
11 OC        Oceania
12 SA        South America
13
14 7 rows selected.
```

Этот запрос выбирает все строки и все столбцы (на что указывает `*` после `SELECT`, что является сокращением от «все столбцы») из таблицы с именем `regions`. Если вы хотите вернуть определенный список столбцов, вы можете указать их отдельно:

```
1 SQL> SELECT name
2     2*   FROM regions;
3
4 NAME
5 -----
6 Africa
7 Antarctica
8 Asia
9 Europe
10 North America
11 Oceania
12 South America
13
14 7 rows selected.
```

SELECT without FROM

SQL standard nevertheless requires a `FROM` clause. Many databases have “dummy” tables to enable such queries, such as the `dual` table in Oracle Database.

```

1 SQL> SELECT 1+2
2       2*   FROM dual;
3
4       1+2
5  -----
6         3

```

Однако многие базы данных, в том числе Oracle Database, ослабили это ограничение стандарта SQL и позволяют в таких случаях не использовать `FROM` в запросах:

```

1 SQL> SELECT 1+2;
2
3       1+2
4  -----
5         3

```

JOIN

The relational model is all about normalizing data, that is putting independent data into separate tables and defining *relationships* between these tables. To recombine normalized data, you can use *joins* to join these tables back together. The following example has two tables: the previously queried `regions` table, and the new `countries` table. To write a query that joins both tables into one result, you use the `JOIN` clause. Without a `JOIN` clause, if you specify the two tables in the `FROM` clause, every row from the table `regions` will be multiplied by every row from the table `countries`. This is often referred to as the *cross product* and it's a common mistake that SQL beginners make. For example:

```

1 SQL> SELECT r.name, c.name
2       2*   FROM regions r, countries c;
3
4  NAME      NAME
5  -----
6  Africa    Kosovo
7  Africa    Yemen
8  Africa    South Africa
9  Africa    Zambia
10 Africa    Zimbabwe
11 Africa    Andorra
12 Africa    United Arab Emirates
13 Africa    Afghanistan
14 Africa    Antigua and Barbuda
15 Africa    Albania
16 Africa    Armenia
17 Africa    Angola
18 Africa    Argentina
19 Africa    Austria

```

```

23  ...
24  South America    Uzbekistan
25  South America    Vatican City
26  South America    Saint Vincent and the Grenadines
27  South America    Venezuela
28  South America    Vietnam
29  South America    Vanuatu
30  South America    Samoa
31
32  1,372 rows selected.

```

Результат этого запроса явно неверный. Не существует ни 1372 стран, ни Австрии в Африке. На самом деле нам нужно объединить все строки из таблицы `countries` с таблицей `regions` где `region_id` идентично. Обычно это называется *условием объединения* и может быть указано в предложении `ON` как часть предложения `JOIN`:

```

1  SQL> SELECT r.name, c.name
2         2 FROM regions r
3         3 JOIN countries c
4         4*      ON (r.region_id=c.region_id);
5
6  NAME      NAME
7  -----
8  Africa    South Africa
9  Africa    Zambia
10 Africa    Zimbabwe
11 Africa    Angola
12 Africa    Burkina Faso
13 Africa    Burundi
14 Africa    Benin
15 ...
16 ...
17 ...
18 South America    Ecuador
19 South America    Guyana
20 South America    Peru
21 South America    Paraguay
22 South America    Suriname
23 South America    Uruguay
24 South America    Venezuela
25
26  196 rows selected.

```

Вот это уже больше похоже на правду! Следует отметить еще одну вещь: в приведенных выше запросах указаны `SELECT r.name, c.name` и помещены буквы `r` и `c` рядом с названиями таблиц. Это *псевдонимы таблиц*, которые необходимы для указания базе данных, какой столбец таблицы вам нужен. Если бы в инструкции просто говорилось `SELECT name, name`, было бы неясно, осится ли запрос к `regions` имени столбца таблицы или к `countries`.ени столбца таблицы.

сих пор вы всегда получали все строки, которые были в таблицах. Если вы хотите вернуть только все страны Южной Америки, используйте условие `WHERE`. Его можно использовать для сопоставления всех строк со значением столбца `regions.name` 'South America':

```

1  SQL> SELECT r.name, c.name
2      2 FROM regions r
3      3 JOIN countries c
4      4 ON (r.region_id=c.region_id)
5      5* WHERE r.name = 'South America';
6
7  NAME                NAME
8  -----
9  South America      Argentina
10 South America      Bolivia
11 South America      Brazil
12 South America      Chile
13 South America      Colombia
14 South America      Ecuador
15 South America      Guyana
16 South America      Peru
17 South America      Paraguay
18 South America      Suriname
19 South America      Uruguay
20 South America      Venezuela
21
22 12 rows selected.

```

GROUP BY

The `GROUP BY` clause is often a puzzle for SQL beginners. The clause is used to aggregate multiple rows into a group, essentially combining multiple rows into one row. When would such a thing ever be useful? Well, for example, the `countries` table contains a column called `population` but the `regions` table does not:

```

1  SQL> SELECT r.name, c.name, c.population
2      2 FROM regions r
3      3 JOIN countries c
4      4 ON (r.region_id=c.region_id)
5      5* WHERE r.name = 'South America';
6
7  NAME                NAME                POPULATION
8  -----
9  South America      Argentina      44694000
10 South America      Bolivia      11306000
11 South America      Brazil      208847000
12 South America      Chile      17925000
13 South America      Colombia      48169000
14 South America      Ecuador      16291000
15 South America      Guyana      741000
16 South America      Peru      31331000
17 South America      Paraguay      7026000
18 South America      Suriname      598000

```

44 | 12 rows selected.

Распространенный бизнес-вопрос: «Какова общая численность населения в каждом регионе?» Учитывая, что в таблице `regions` нет столбца с этой информацией, ответ можно получить, только сложив значения в столбце `population` для каждой страны по регионам. Таким образом, вам нужен механизм, который возьмет 196 строк из таблицы `countries` и разделит их на семь групп или категорий в зависимости от региона (поскольку в таблице `regions` семь строк). Однако запрос не может просто преобразовать 196 строк в семь строк. Необходимо рассчитать численность населения по регионам на основе численности населения стран, входящих в регион. Это можно сделать, применив функцию `SUM()` `aggregate` к столбцу `population`:

```

1  SQL> SELECT r.name, SUM(c.population)
2      2   FROM regions r
3      3  JOIN countries c
4      4      ON (r.region_id=c.region_id)
5      5* GROUP BY r.name;
6
7  NAME                                SUM(C.POPULATION)
8  -----
9  Africa                             1263685000
10 Asia                               4439011000
11 Europe                             748985000
12 North America                      575767000
13 Oceania                            37556000
14 South America                      421986000
15
16  6 rows selected.
```

Этот запрос показывает еще кое-что интересное. Несмотря на то, что в таблице `regions` семь регионов, запрос выдал шесть строк. Это связано с тем, что есть регион `'Antarctica'`, но в таблице `region_id` нет страны с таким `countries` названием. Таким образом, условие `JOIN` отфильтровывает этот регион (поскольку в таблице `region_id` нет соответствующего `countries` названия, указанного в условии `ON`). Условие `GROUP BY` не требует наличия условия `JOIN`; группы можно создавать только в одной таблице. Например, на вопрос «Сколько стран начинаются с одной и той же буквы?» тоже можно ответить с помощью `GROUP BY`. Для этого создайте столько групп, сколько существует уникальных значений первой буквы для всех строк, используя функцию `SUBSTR()`, а затем посчитайте строки, которые попадают в эту группу или категорию:

```

1  SQL> SELECT SUBSTR(name,1,1), COUNT(*)
2      2   FROM countries
3      3* GROUP BY SUBSTR(name,1,1);
4
5  SUBSTR(NAME,1,1)    COUNT(*)
```

```

9      S      20
10     Z      2
11     A     11
12     U      7
13     B     17
14     C     17
15     D      5
16     G     11
17     E      8
18     F      3
19     M     18
20     H      3
21     I      8
22     J      3
23     N     11
24     L      9
25     O      1
26     P      9
27     Q      1
28     R      3
29     T     12
30     V      4
31
32 24 rows selected.

```

HAVING

The **HAVING** clause filters rows resulting from the **GROUP BY** clause based on the predicate(s) provided. For example, if you want to return only the regions that have a population of more than 500 million people, this cannot be specified in the **WHERE** clause because the **WHERE** clause is processed before the **GROUP BY** clause. Hence, the **WHERE** clause has no notion of the population of a region. This is where the **HAVING** clause comes in. From a logical perspective, it behaves the same as the **WHERE** clause, but it filters at a different processing stage:

```

1  SQL> SELECT r.name, SUM(c.population)
2         FROM regions r
3         JOIN countries c
4         ON (r.region_id=c.region_id)
5         GROUP BY r.name
6         HAVING SUM(c.population) > (500 * 1000 * 1000);
7
8  NAME                                SUM(C.POPULATION)
9  -----
10 Africa                             1263685000
11 Asia                               4439011000
12 Europe                             748985000
13 North America                      575767000

```

ORDER BY

BY clause can be used to return the rows in alphabetical order.

```

1  SQL> SELECT SUBSTR(name,1,1), COUNT(*)
2      2 FROM countries
3      3 GROUP BY SUBSTR(name,1,1)
4      4* ORDER BY SUBSTR(name,1,1);
5
6  SUBSTR(NAME,1,1)      COUNT(*)
7  -----
8  A                      11
9  B                      17
10 C                      17
11 D                       5
12 E                       8
13 F                       3
14 G                      11
15 H                       3
16 I                       8
17 J                       3
18 K                       6
19 L                       9
20 M                      18
21 N                      11
22 O                       1
23 P                       9
24 Q                       1
25 R                       3
26 S                      26
27 T                      12
28 U                       7
29 V                       4
30 Y                       1
31 Z                       2
32
33 24 rows selected.

```

По умолчанию строки сортируются по возрастанию, но вы можете изменить порядок на противоположный с помощью ключевого слова `DESC` (DESCENDING):

```

1  SQL> SELECT SUBSTR(name,1,1), COUNT(*)
2      2 FROM countries
3      3 GROUP BY SUBSTR(name,1,1)
4      4* ORDER BY SUBSTR(name,1,1) DESC;
5
6  SUBSTR(NAME,1,1)      COUNT(*)
7  -----
8  Z                      2
9  Y                      1
10 V                      4
11 U                      7
12 T                      12
13 S                      26
14 R                      3

```

```

18      N
19      M
20      L
21      K
22      J
23      I
24      H
25      G
26      F
27      E
28      D
29      C
30      B
31      A
32
33      24 rows selected.

```

OFFSET

The `OFFSET` clause specifies the number of rows to skip before starting to return data. This clause is shorthand for what would otherwise require analytic queries or subqueries. For example, asking “Give me all countries in South America ranked by square kilometers except the first three” can be answered with:

```

1      SQL> SELECT c.name, c.area_sq_km
2          2      FROM countries c
3          3      JOIN regions r
4          4          ON (c.region_id=r.region_id)
5          5      WHERE r.name = 'South America'
6          6      ORDER BY area_sq_km DESC
7          7*      OFFSET 3 ROWS;
8
9      NAME                AREA_SQ_KM
10     -----
11     Colombia            1138910
12     Bolivia             1098581
13     Venezuela           912050
14     Chile                756102
15     Paraguay            406752
16     Ecuador             283561
17     Guyana              214969
18     Uruguay             176215
19     Suriname            163820
20
21      9 rows selected.

```

FETCH

The `FETCH` clause specifies the number of rows to return from the result. Some databases call this the `LIMIT` clause. Like the `OFFSET` clause, this is also a shorthand and can be used to answer business questions like “What are the top three ntries in terms of population?” This can be answered with:

4	4*	FETCH FIRST 3 ROWS ONLY;
5		
6	NAME	POPULATION
7	-----	-----
8	China	1384689000
9	India	1296834000
10	United States	329256000

Вы можете задаться вопросом, что произойдет, если две строки будут иметь одинаковое значение в третьей позиции. Будут ли возвращены обе строки? Или только первая? Для таких случаев в предложении `FETCH` предусмотрены ключевые слова `ONLY` и `WITH TIES`. В приведенном выше примере используется `ONLY`, поскольку маловероятно, что у двух стран будет одинаковое население. Однако при ранжировании стран по первой букве алфавита вероятность совпадения гораздо выше. Например, в примере с ранжированием стран по первой букве алфавита по количеству стран очевидно, что у некоторых букв одинаковое количество стран:

```

1  SQL> SELECT SUBSTR(name,1,1), COUNT(*)
2      2      FROM countries
3      3      GROUP BY SUBSTR(name,1,1)
4      4*      ORDER BY COUNT(*) DESC;
5
6  SUBSTR(NAME,1,1)      COUNT(*)
7  -----
8  S                      26
9  M                      18
10 B                      17
11 C                      17
12 T                      12
13 A                      11
14 N                      11
15 G                      11
16 L                      9
17 P                      9
18 I                      8
19 E                      8
20 U                      7
21 K                      6
22 D                      5
23 V                      4
24 J                      3
25 H                      3
26 F                      3
27 R                      3
28 Z                      2
29 Q                      1
30 Y                      1
31 O                      1
32
3  24 rows selected.

```

```

1  SQL> SELECT SUBSTR(name,1,1), COUNT(*)
2      2 FROM countries
3      3 GROUP BY SUBSTR(name,1,1)
4      4 ORDER BY COUNT(*) DESC
5      5*  FETCH FIRST 3 ROWS ONLY;
6
7  SUBSTR(NAME,1,1)      COUNT(*)
8  -----
9  S                      26
10 M                      18
11 B                      17

```

Здесь в дело вступает ключевое слово `WITH TIES`, которое включает в результаты совпадения:

```

1  SQL> SELECT SUBSTR(name,1,1), COUNT(*)
2      2 FROM countries
3      3 GROUP BY SUBSTR(name,1,1)
4      4 ORDER BY COUNT(*) DESC
5      5*  FETCH FIRST 3 ROWS WITH TIES;
6
7  SUBSTR(NAME,1,1)      COUNT(*)
8  -----
9  S                      26
10 M                      18
11 B                      17
12 C                      17

```

OFFSET and FETCH

Combining the `OFFSET` and `FETCH` clauses allow another nice shorthand that would otherwise require an analytical query or subquery. Consider the question “What is the second smallest country on the planet in terms of square kilometers?” This can be answered by combining `OFFSET` to return results starting from the second row, and `FETCH` to fetch just the second row:

```

1  SQL> SELECT name, area_sq_km
2      2 FROM countries
3      3 ORDER BY area_sq_km
4      4 OFFSET 1 ROW
5      5*  FETCH FIRST 1 ROW ONLY;
6
7  NAME      AREA_SQ_KM
8  -----
9  Monaco      2

```

What's Next?

ORACLE

Oracle offers a wide range of technologies for building, testing, and maintaining applications in the cloud and in your data center. Find free tools and learning resources at oracle.com/developer

[Learn More →](#)

TRENDING STORIES

1. [Timescale Hyperfunctions Ease PostgreSQL Query Complexities](#)
2. [Maria DB Gets Reactive with a Non-Blocking Connector for Java](#)
3. [6 Tips for Better Data Science in the Cloud](#)
4. [Distributed SQL Takes Databases to the Next Level](#)
5. [Alation Delivers a New Data Governance App](#)

TNS



Gerald Venzl has worked as a developer, DBA, performance engineer, software architect, consultant and enterprise architect. He is still active as a developer in his free time and on open source projects. Gerald is currently VP, developer initiatives, Oracle Database.

[Read more from Gerald Venzl →](#)

SHARE THIS STORY



TRENDING STORIES

[Timescale Hyperfunctions Ease PostgreSQL Query Complexities](#)

4. Distributed SQL Takes Databases to the Next Level

5. Alation Delivers a New Data Governance App

INSIGHTS FROM OUR SPONSOR

ORACLE

Oracle offers a wide range of technologies for building, testing, and maintaining applications in the cloud and in your data center. Find free tools and learning resources at oracle.com/developer

[Learn More →](#)

TNS DAILY NEWSLETTER

Receive a free roundup of the
most recent TNS articles in
your inbox each day.

SUBSCRIBE

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap



© The New Stack 2026

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)